Token & Costi

Contare, stimare, ottimizzare le chiamate agli LLM

Modulo M2 · Lezione 14 · Daniele Martini · 25 giugno 2026



Quanto è costata la vostra prima chiamata?

Sabato avete chiamato un LLM per la prima volta. In dev i costi sono invisibili; in **produzione**, a migliaia di chiamate, sono una voce di bilancio. Stasera la copriamo in tre mosse.

1 · Contare

Cos'è un token e con quale tokenizer si conta — diverso per ogni provider.

2 · Stimare

Da token a euro: input vs output, conversazioni multi-turn, costi nascosti.

3 · Ottimizzare

Le leve per tagliare la spesa senza rovinare la qualità.

- Capire i token spiega anche limiti (context window), latenza e — più avanti — il costo del RAG.
- Ci agganciamo al pricing comparison già visto il 23/06 (LLM APIs Deep Dive).



1. Token e tokenizer



Un token non è una parola

Né una parola né un carattere: è un **pezzo di testo** (*subword*) prodotto da un algoritmo. Le parole comuni diventano un token solo; quelle rare si spezzano in più pezzi.

Come si spezza il testo

`token` `ization` · `the`

`tokenization` → 2 token · `the` → 1 token · una parola rara → 3–4 token.

Perché ci importa

- Non si contano caratteri o parole: si contano i **token**.
- Il token è l'unità di prezzo *e* l'unità del context window.
- Quanti token servono dipende dall'algoritmo di tokenizzazione.

Già visto in Intro GenAI ad alto livello — ora lo rendiamo misurabile.



Come funziona BPE

Quasi tutti i tokenizer moderni usano **Byte Pair Encoding**. L'idea: partire dai singoli caratteri e **fondere ripetutamente la coppia più frequente**, finché il vocabolario raggiunge la dimensione voluta.

L'algoritmo in 4 passi

1. Spezza il testo nei singoli caratteri.
2. Conta tutte le coppie di simboli adiacenti.
3. Fondi la coppia **più frequente** in un nuovo simbolo.
4. Ripeti finché il vocabolario è grande quanto serve.

Esempio su un corpus giocattolo

```
parti da: l o w · l o w e r · n e w e s t · w i d e s t
▸ merge 1: "e s" → es (la coppia più frequente)
n e w es t · w i d es t
▸ merge 2: "es t" → est
l o w · l o w e r
▸ merge 3: "l o" → lo … e così via
```

Caratteri singoli → Fondi le coppie più frequenti → **Vocabolario di subword**



BPE: perché conta per i token

Le sequenze frequenti diventano **un token solo**; quelle rare restano spezzate in più pezzi. È questo che decide quanti token — e quindi quanto — paghi.

La stessa idea, su parole vere

`the`

parola comune → **1 token**

`token` `ization`

composta → **2 token**

`tok` `enizz` `azione`

rara / in italiano → **3+ token**

Split illustrativi: il numero esatto dipende dal tokenizer del modello.

Che problema risolve

Un compromesso tra **dimensione del vocabolario** (non un token per ogni parola possibile) e **lunghezza delle sequenze** (non un token per carattere).

Conseguenza pratica

Più un testo somiglia ai dati di training (inglese, codice comune), **meno token** serve. Testo raro o in altre lingue → più token → più costo.



Ogni provider ha il suo tokenizer

Lo stesso testo conta token diversi a seconda del modello: non esiste un costo "in generale". Si conta sempre col tokenizer di chi userai.

OpenAI

Libreria `tiktoken`, open source: conti i token **in locale**, senza chiamare l'API.

```
enc = tiktoken.encoding_for_model("gpt-4o")
enc.encode("testo")
```

Encoding per modello:
`cl100k_base` (GPT-4/3.5)
`o200k_base` (GPT-4o+)

Gemini

Tokenizer **locale** (SentencePiece) via Vertex AI: conti i token offline, **senza API key**.

```
get_tokenizer_for_model(
  "gemini-1.5-flash").count_tokens( ... )
```

Conteggio diverso da OpenAI sullo stesso testo.

DeepSeek

Tokenizer proprio, distribuito **open** su Hugging Face: lo carichi e conti i token in locale.

```
AutoTokenizer.from_pretrained(
  "deepseek-ai/DeepSeek-V3")
```

Vocabolario e conteggi diversi sia da OpenAI sia da Gemini.

- Tre tokenizer, tre conteggi diversi sullo **stesso** testo.
- Non assumere mai parità tra provider: conta col tokenizer giusto.



Demo — stesso testo, tre righelli

DEMO DAL VIVO

"Gli LLM tokenizzano il testo in subword."

OpenAI · o200k_base

?

e cl100k_base dà un numero ancora diverso

Gemini · 2.5 Flash

?

tokenizer locale via Vertex AI

DeepSeek · V3

?

tokenizer proprio via HF

I conteggi non coincidono. E in italiano salgono ancora — lo vediamo ora.



La lingua conta (in token)

I tokenizer sono allenati soprattutto sull'inglese: l'inglese si comprime meglio. La **stessa frase** tradotta usa più token in italiano.

English

Meno token a parità di significato: la lingua su cui i tokenizer sono più efficienti.

Italiano

Più token per la stessa frase → **lo stesso contenuto costa di più**.

Il fattore esatto varia con la frase: lo mostriamo a video, non lo fissiamo a percentuale.



Il limite è in token

Il **context window** (128k, 200k, fino a 1M) è espresso in token, non caratteri. Tutto ciò che entra compete per lo stesso budget.



Un solo budget condiviso

System prompt, history, documenti RAG e output sommano sullo stesso limite. Quando è pieno, qualcosa deve uscire.

Gancio al RAG (M3)

Più chunk recuperi, meno spazio (e più costo) per il resto. Il RAG va dimensionato in token.

2. Calcolo dei costi



Prezzo per token: input ≠ output

INPUT

Tutto ciò che mandi, a **ogni** chiamata:

- System prompt
- History della conversazione
- Few-shot examples
- Nuovo messaggio

OUTPUT

4–6×

Più caro dell'input. Conseguenze pratiche:

- Una risposta lunga costa più di un prompt lungo
- È il token che **controlli** con `max_tokens`
- Tagliarlo è la prima leva di risparmio

Prezzo quotato per 1M token. Input e output hanno listini separati.



Quanto costano davvero

Listino di riferimento (giugno 2026, $ per 1M token, input / output). Ripreso dal confronto del 23/06.

OpenAI

GPT-5.5	5 / 30
GPT-5.4	2,5 / 15
GPT-5.4 Nano	0,20 / 1,25

Gemini

2.5 Pro	1,25 / 10
2.5 Flash	0,30 / 2,50

DeepSeek (il più economico)

DeepSeek-V3	0,27 / 1,10
DeepSeek-R1	0,55 / 2,19

- Dal più economico (DeepSeek) al flagship: **più di un ordine di grandezza** di differenza.
- I listini cambiano spesso: verifica la pagina ufficiale, non hardcodare i prezzi.



Quanto costa una chiamata

```
1.000 token input
  500 token output
  modello: GPT-5.4 ($2,50 / $15,00)

input  = 1.000/1e6 × $2,50  = $0,0025
output =   500/1e6 × $15,00 = $0,0075
```

Output a metà dei token dell'input, ma **pesa di più**: è il listino separato in azione.

totale

$0,01

a chiamata.

Sembra nulla. Ma una **conversazione** non è una chiamata sola…



Le API sono stateless

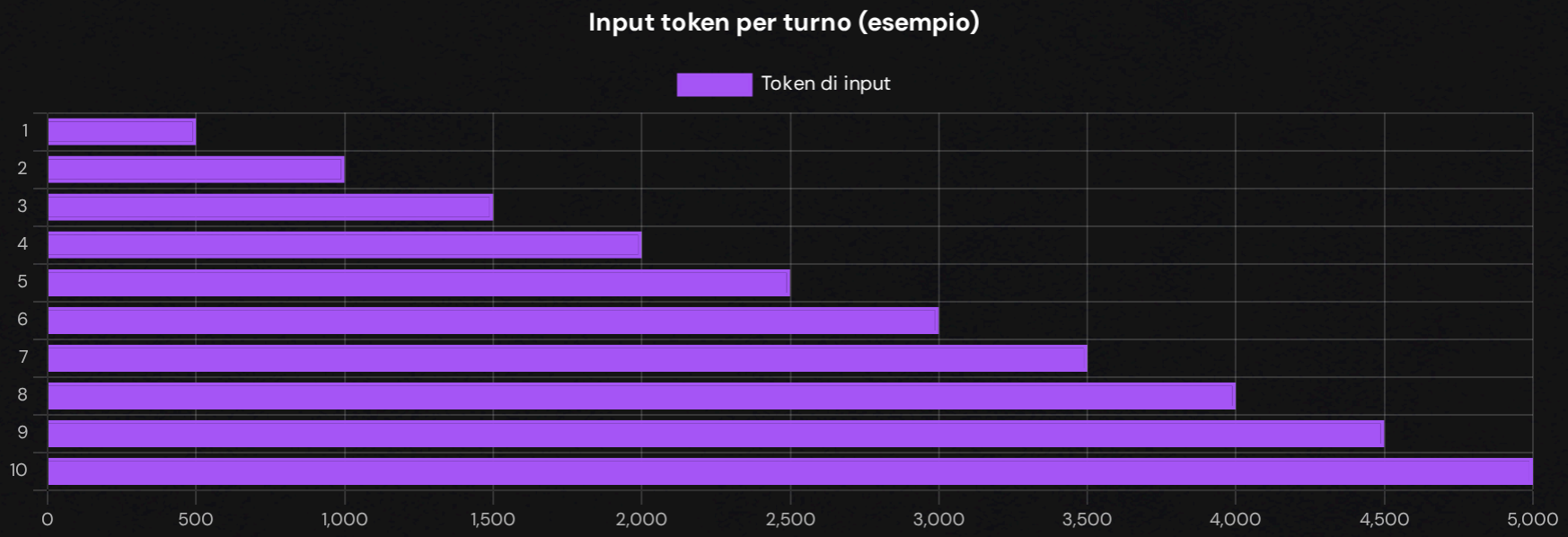
Il modello non ricorda nulla tra una chiamata e l'altra: a ogni turno **ri-invii tutta la conversazione** come input. La domanda nuova è corta, ma trascini tutta la history.



Il costo per turno cresce a ogni scambio, anche se la nuova domanda è cortissima.

Dalla domanda 1 alla domanda 10

Input token per turno: alla decima domanda re-invii ~10× rispetto alla prima. La spesa è quasi tutta **history re-inviata**, non testo nuovo.



Per questo conviene gestire la history con memory/summary, invece di re-inviare tutto.

Lineare per messaggio, quadratico in totale

Poiché ogni turno re-invia tutta la history, il costo del **singolo messaggio** cresce **linearmente** con la lunghezza della conversazione. Ma il costo **totale** è la somma di tutti i turni: cresce **quadraticamente**.

Costo del turno *n*

$\approx n \cdot c$ → lineare

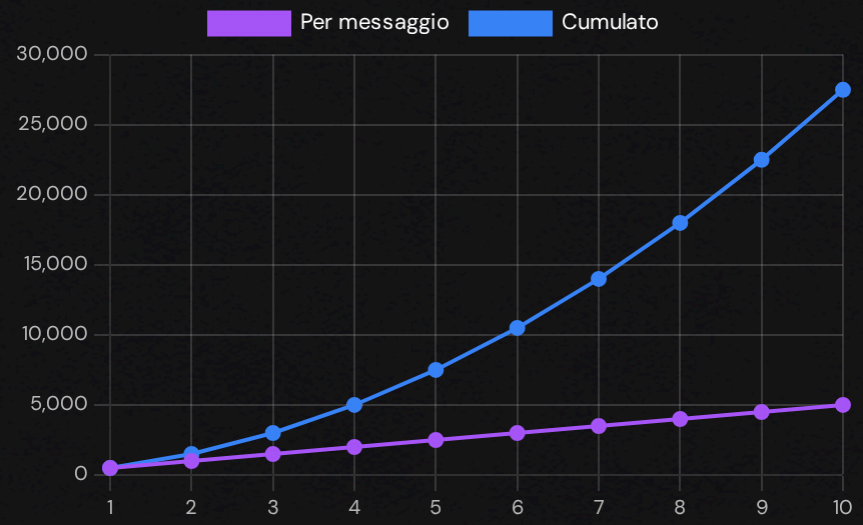
ogni turno trascina tutta la conversazione precedente.

Costo totale (*N* turni)

$c \cdot (1+2+\ldots+N) = c \cdot N(N+1)/2$ → $\propto N^2$

la somma dei turni è una progressione: quadratica in N.

Token di input — per messaggio vs cumulati (c≈500)



Raddoppi la lunghezza della conversazione → il costo totale quadruplica (×4).

Cosa paghi senza accorgertene

Tre voci viaggiano a **ogni** chiamata e gonfiano l'input senza che te ne accorga.

System prompt

1 mld

Un system da 1.000 token × 1M chiamate = 1 miliardo di token di **sole istruzioni**, ripagati ogni volta.

Few-shot examples

Ogni esempio nel prompt è input pagato a ogni chiamata. 5 esempi lunghi = un costo fisso su **tutte** le richieste.

RAG context (M3)

I chunk recuperati finiscono nell'input: più documenti dai al modello, più paghi. Da progettare con cura.

- Il costo non è solo "la domanda": è tutto il contorno che si ripete.
- Caching e scelta del modello (Cap 3) servono proprio a tagliare queste voci.



Leve "gratis": caching e batch

Prima ancora di riscrivere i prompt, i provider offrono due sconti che riducono il costo a parità di contenuto.

Prompt caching — -90%

La parte stabile del prompt (system, esempi) viene messa in cache e **riletta a costo ridotto**. OpenAI -90% sull'input cached; Gemini context caching sconta i token ripetuti.

Batch API — -50%

Per lavori **non in tempo reale** (elaborazioni offline, dataset): inviati in batch a circa metà prezzo.

Sconti reali, a parità di prompt — le riprendiamo tra le leve del Cap 3.



3. Ottimizzazione per costo



Scegli il modello giusto

La leva **più potente**, in assoluto. Non tutto richiede il flagship: lo stesso task, su modelli diversi, cambia di due ordini di grandezza.

$0,27

DeepSeek-V3 · input /1M

classificazione, estrazione, routing semplice

$5,00

GPT-5.5 · input /1M

ragionamento complesso, task difficili

- **Model tiering**: usa il modello più piccolo che supera la soglia di qualità accettabile.
- Spesso un task "semplice" sul flagship è soldi buttati: provalo prima su un modello economico.



Taglia i token di input

L'input è tutto ciò che mandi: ridurlo, senza perdere informazione utile, abbassa il costo di ogni chiamata.

Prompt concisi

Una istruzione per cosa, niente ripetizioni o preamboli inutili. Le istruzioni ridondanti si pagano a ogni chiamata.

Comprimi i few-shot

Togli gli esempi quando il modello non ne ha bisogno, o riducine numero e lunghezza. Spesso 2 buoni bastano.

Gestisci la history

Finestra mobile o riassunto dei turni vecchi, invece di re-inviare tutta la conversazione. → memory / summary

È qui che la conversazione multi-turn smette di esplodere di costo.



Controlla l’output (è il più caro)

L'output costa 4–6× l'input: ogni token di risposta in meno è il risparmio che pesa di più.

`max_tokens`

Metti un tetto duro alla risposta: evita generazioni che vanno lunghe senza motivo.

Chiedi risposte brevi

"Rispondi in una frase" costa una **frazione** di "spiega in dettaglio". Dillo esplicitamente nel prompt.

Structured output

JSON / schema: il modello riempie i campi e basta, niente giri di parole.
→ lezione Structured Output

Più corta e strutturata è la risposta, meno costa — spesso senza perdere valore.



Sfrutta il caching

Dividi il prompt in due: ciò che resta uguale tra le chiamate va in cache e si paga pochissimo.

🔒 **Parte stabile**
system prompt, esempi, documenti fissi — sempre uguali

costo ridotto

Parte variabile
la domanda dell'utente — cambia ogni volta

prezzo pieno

- Metti in **testa** al prompt ciò che si ripete uguale: è la condizione per il cache hit.
- Conviene solo se la parte stabile è grande e davvero ripetuta tra le chiamate.



Caching: automatico o da attivare?

Stessa idea (pagare pochissimo il prefisso ripetuto), ma **come si ottiene** dipende dal provider: c'è chi lo fa da solo e chi te lo fa attivare.

Automatico (implicito)

Nessuna modifica al codice: il provider riconosce da solo il prefisso ripetuto e applica lo sconto.

OpenAI, **DeepSeek**, **Gemini 2.5** lo fanno di default.

Scatta sopra una soglia di lunghezza (es. OpenAI ~1.024 token).

Esplicito (lo attivi tu)

Dichiari **cosa** mettere in cache e **per quanto**, e poi la riusi.

Es. **Gemini** `CachedContent` : crei la cache, la richiami via handle, con un TTL e un piccolo costo di storage.

Conviene per contesti grandi (documenti, system lungo) riusati spesso.

- Perché scatti il **cache hit**: prefisso in **testa**, **identico** tra le chiamate (un carattere diverso = cache miss) ed entro il **TTL** (pochi minuti, poi scade).



Non ottimizzare alla cieca

Ogni leva ha un prezzo nascosto: tre forze da bilanciare, non un singolo numero da minimizzare.

Costo

Quello che stiamo tagliando — ma non è l'unica metrica.

Qualità

Tagliare troppo l'input o forzare output minimi la degrada; il modello economico può non bastare.

Latenza

Modello, lunghezza output e batch spostano anche i tempi di risposta.

La regola: misurare, non indovinare.



Asciutto costa meno, esplicito rende meglio

Un system prompt più corto consuma meno token a ogni chiamata. Ma spesso **spiegare meglio paga**: istruzioni ed esempi chiari migliorano l'output. Il taglio giusto non è il più aggressivo.

System prompt asciutto

+ meno token → meno costo a *ogni* chiamata.

- se togli istruzioni o esempi utili, la qualità cala: più errori, risposte fuori formato, più ritentativi (che ri-costano).

System prompt esplicito

+ output più centrato, meno iterazioni e meno correzioni a valle.

- costo fisso più alto per chiamata, pagato sempre.

Spesso un prompt chiaro che "azzecca" la risposta al primo colpo costa **meno in totale**.

Taglia la ridondanza, non l'informazione utile: misura la qualità, non solo i token.



Asciugare il prompt: prima e dopo

Togliere **solo** la ridondanza (preamboli, ripetizioni) lascia intatte le istruzioni essenziali e taglia i token a parità di risultato.

Prima

Prompt verboso: preamboli, ripetizioni, istruzioni ridondanti.

1.200 tok

Dopo

Asciutto: stesse istruzioni essenziali, zero ridondanza.

400 tok

Stesso compito, −67% di token

- Misura prima/dopo con `tiktoken` (numeri illustrativi: fissarli in prep).
- −800 token/chiamata × **100.000 chiamate/mese** = risparmio reale, ricorrente.



Quando il costo conta davvero

Per un uso individuale o un prototipo il costo è quasi sempre **trascurabile**: parliamo di centesimi. Non ottimizzare prematuramente. Il costo diventa un problema vero su due assi.

Uso individuale / sviluppo — poche chiamate al giorno: il costo è rumore di fondo. Concentrati sul far funzionare le cose, non sui centesimi.

Scala — tanti utenti / volumi

Il costo per chiamata, anche minuscolo, va moltiplicato per **milioni** di richieste. È qui che caching, model tiering e prompt asciutti ripagano.

Flussi token-heavy su modelli costosi

Context lunghi, RAG con molti chunk, agenti multi-step su un **flagship**: ogni singola chiamata è già cara, e se ne fanno tante.

Ottimizza dove il costo si moltiplica — volume × costo per chiamata — non prima.



Contare → Stimare → Ottimizzare

Il filo della serata, in tre mosse e tre cose da portare a casa.

Contare

Il conteggio è **provider-specifico**: usa il tokenizer del modello, mai assumere parità.

Stimare

Input vs output, history multi-turn, costi nascosti: sai da dove arriva la spesa.

Ottimizzare

Modello giusto, input ridotto, output controllato, caching: in quest'ordine di impatto.

L'output costa più dell'input — e la history multi-turn è il costo nascosto numero uno.



La lente resta accesa

La cost-awareness di stasera non finisce qui: ritorna, sempre più pesante, lungo tutto il corso.

Gestione della history

prossima

Memory e summary dei turni vecchi, per non re-inviare tutta la conversazione.

RAG Modulo 3

Quanti chunk diamo al modello? Ogni documento in più è input pagato.

Production Modulo 5

Monitoring e cost management: tenere il costo sotto controllo in esercizio.

Contare, stimare, ottimizzare: una lente da tenere accesa per sempre.



Grazie

Contare → Stimare → Ottimizzare

Prossimi passi: gestire la conversazione (memory & summary) · RAG (M3) · produzione (M5)

